

Data Modeling – Slowly Changing Dimensions (SCDs)

Day 2 Lab Follow-Along Notes – DataExpert.io Free Boot Camp

Video: Data Modeling - Building Slowly Changing Dimensions (SCDs) - Day 2 Lab Instructor: Data With Zach / DataExpert.io

1. What Is the Goal of This Lab?

The Day 2 lab focuses on:

- Slowly Changing Dimensions (SCDs)
- Historical tracking
- State changes over time
- Building analytical history tables
- Detecting changes incrementally

The lab builds a player dimension table that tracks how player attributes evolve over time.

2. Core Problem Being Solved

Suppose a player's attributes change over time:

player_name	scoring_class
Kobe Bryant	good
Kobe Bryant	star

If we overwrite the old value:

star replaces good

we lose history.

But analytics often need:

- what the value was at a specific time

- when it changed
- how long the state lasted

This is the exact problem SCDs solve.

3. What Is a Slowly Changing Dimension?

A Slowly Changing Dimension (SCD) tracks changes to dimensional attributes over time.

Examples:

- customer address changes
- employee department changes
- subscription plan changes
- player scoring category changes
- account status changes

The word “slowly” means:

changes happen occasionally, not every second

4. Types of SCDs

Type 1

Overwrite old values.

No history preserved.

Example:

player	scoring_class
Kobe	star

Previous states are gone.

Type 2 (Main Focus)

Create NEW rows when values change.

History is preserved.

Example:

player	scoring_class	start_season	end_season
Kobe	good	1996	1999
Kobe	star	2000	2005

This is what the lab builds.

5. Key Insight of SCD Type 2

Instead of storing:

current value only

we store:

periods of validity

Each row represents:

"This attribute value was true during this time range."

6. Main Table Structure

The lab creates an SCD history table.

Typical structure:

```
CREATE TABLE players_scd (  
  player_name TEXT,  
  scoring_class scoring_class,  
  is_active BOOLEAN,  
  start_season INTEGER,  
  end_season INTEGER,
```

```
current_season INTEGER  
);
```

7. Understanding the Columns

player_name

Entity identifier.

scoring_class

Categorical player quality.

Example:

- star
 - good
 - average
 - bad
-

is_active

Tracks whether player currently plays.

start_season

When this state began.

end_season

When this state stopped being true.

current_season

Snapshot build year.

8. Biggest Concept – State Transition Tracking

The entire lab is about:

detecting when attributes change

If attributes remain the same:

✓ extend existing period

If attributes change:

✓ close old row ✓ create new row

This is the heart of SCD Type 2.

9. Example Walkthrough

Suppose:

season	scoring_class
1996	average
1997	average
1998	good
1999	good
2000	star

SCD output becomes:

player	class	start	end
player	average	1996	1997
player	good	1998	1999
player	star	2000	2000

Notice:

- repeated states are compressed

- only transitions create new rows

Huge optimization.

10. Why This Is Powerful

Without SCD compression:

player	season	class
Kobe	1996	average
Kobe	1997	average
Kobe	1998	good
Kobe	1999	good

Many repeated rows.

With SCD:

class	start	end
average	1996	1997
good	1998	1999

Much smaller history representation.

11. Incremental Pipeline Logic

The pipeline compares:

Yesterday

Previous SCD state.

AND

Today

Current season snapshot.

Then decides:

- unchanged
- changed
- new player
- retired player

12. Core SQL Thinking

This lab is NOT just SQL.

It is teaching:

```
state machine thinking
```

Every player transitions between states over time.

The warehouse stores those transitions.

13. Detecting Changes

Key condition:

```
current_attribute <> previous_attribute
```

Example:

```
t.scoring_class <> y.scoring_class
```

This indicates:

```
state transition happened
```

14. The Four Main Cases

Case 1 – New Player

Player exists today but not yesterday.

Action:

☒ create new row

Case 2 – No Change

Attributes unchanged.

Action:

☒ extend end_season

Case 3 – Changed Attributes

Example:

average -> good

Action:

☒ close old row ☒ insert new row

Case 4 – Inactive/Retired

Player absent today.

Action:

☒ mark inactive ☒ extend inactivity history

15. Important Modeling Insight

SCD Type 2 tracks:

attribute history

NOT event history.

This distinction is very important.

16. Event Tables vs SCD Tables

Event Table

Stores:

what happened

Example:

| player | season | pts |

SCD Table

Stores:

what state was true during a period

Example:

| player | class | start | end |

17. Why Warehouses Love SCDs

Because analysts constantly ask:

- What was true at this time?
- When did this change?
- How long did this state last?
- What was the customer status then?

SCDs answer these efficiently.

18. Compression Benefit

If a value remains unchanged for 10 years:

Traditional model:

10 rows

SCD model:

1 row with start/end dates

Very powerful compression.

19. Real Industry Examples

Customer Address History

| customer | address | start | end |

Employee Department Tracking

| employee | department | start | end |

Subscription Plans

| user | plan | start | end |

Risk Classification

| customer | risk_level | start | end |

20. Important SQL Concepts Used

FULL OUTER JOIN

Combines yesterday and today.

Needed to detect:

- additions
 - deletions
 - unchanged rows
 - changed rows
-

CASE Statements

Used for state logic.

Example:

```
CASE
WHEN changed THEN ...
WHEN new THEN ...
ELSE ...
END
```

COALESCE

Handles NULLs.

Example:

```
COALESCE(today.player_name, yesterday.player_name)
```

UNION ALL

Very important.

Often used to:

- preserve old closed records
 - append new active records
-

21. Key SCD Design Principle

SCD tables are usually:

```
append-only
```

instead of constantly overwriting history.

This preserves auditability.

22. Current Record vs Historical Record

Common pattern:

Historical rows

| start | end |

with bounded periods.

Current row

Often:

```
end_date = NULL
```

OR latest season.

Indicates currently active state.

23. Common Interview Questions

What is SCD Type 2?

A historical modeling technique that creates new rows whenever tracked attributes change.

Why use SCDs?

To preserve historical state changes over time.

Difference between Type 1 and Type 2?

Type 1	Type 2
overwrite	preserve history
no history	historical tracking
simple	analytical

What are start/end dates used for?

They define the validity period of a dimension state.

24. Most Important Mental Model

Event Data

```
What happened?
```

Snapshot Data

What did the world look like then?

SCD Data

How did entity state evolve?

This lab focuses on the third one.

25. Connection to Day 1

Day 1

Focused on:

cumulative historical storage

using:

- arrays
 - structs
 - snapshots
-

Day 2

Focused on:

tracking state transitions efficiently

using:

- start/end periods
- change detection
- SCD Type 2

Together these form foundational warehouse modeling concepts.

26. Analytical Benefits of SCDs

Enables:

- historical reporting
 - cohort analysis
 - customer lifecycle analysis
 - temporal joins
 - point-in-time reconstruction
 - churn analysis
 - historical segmentation
-

27. Important Real-World Concept

Modern warehouses often combine:

- snapshots
- event streams
- SCD dimensions
- cumulative tables

into layered analytical systems.

This boot camp is teaching those foundations.

28. Practical Intuition

Think of SCD Type 2 as:

`Git history for business entities`

Instead of deleting old versions:

✓ preserve every meaningful state transition.

29. Common Beginner Mistake

Beginners often think:

more rows = more history

But SCDs show:

history can be compressed intelligently

by storing only transitions.

30. Final Big Takeaway

Day 1 taught:

Store history compactly.

Day 2 taught:

Track how state changes over time.

Combined Understanding

Modern data modeling is fundamentally about:

- time
- state
- transitions
- efficient historical analytics

That is the central idea behind cumulative modeling and SCDs.